Predicting GitHub Issue Resolution Time with Repository Signals and Semantic Representations

Eric Huang [1] **Bruce Zhang** [1]

Abstract

GitHub issue resolution-time prediction is an important task for software engineering analytics, supporting workload prioritization and repository management at scale. However, issue resolution behavior is highly heterogeneous across repositories, and it remains unclear how much predictive value comes from repository activity signals versus semantic information embedded in issue discussions.

In this work, we study issue resolution-time prediction on 8.69 million GitHub issues collected from GH Archive under a temporal evaluation setting. We compare three regimes: (1) TF-IDF and repository-signal baselines using logistic regression, (2) transformer-based contextual text models built on DeBERTa-v3, and (3) a hybrid semantic-augmentation framework that prepends LLM-generated summaries to issue text prior to transformer encoding.

Our results show that fine-tuned transformer representations outperform shallow lexical baselines, while repository-level signals provide modest gains due to sparse coverage. Contrary to our initial expectation, LLM-generated semantic summaries improve interpretability but do not improve predictive performance over the raw-text DeBERTa baseline. Further analysis suggests that repository activity, issue length, and contributor status are meaningful filing-time signals, whereas Medium/Slow resolution tiers remain difficult to distinguish under the current label scheme.

1. Introduction

Online distributed version control systems such as GitHub have become central infrastructure for modern collaborative software development. Large open-source repositories on GitHub may contain tens of thousands of issues involving bug reports, feature requests, maintenance discussions, and developer coordination. Predicting issue resolution time is therefore an important task for software engineering analytics, as it may help developers prioritize workloads, estimate maintenance effort, and improve project management efficiency.

However, in practice, prior work has highlighted substantial heterogeneity and bias in GitHub repository data, suggesting that simple repository-level statistics may be insufficient for understanding collaborative development behavior (Kalliamvakou et al., 2014). Study on pull-based software development has shown that repository interaction dynamics, contributor behavior, and discussion processes substantially affect software development outcomes such as pull request acceptance and processing time (Gousios et al., 2014). This motivates semantic modeling of issue discussions through transformer-based language representations.

GitHub issues are fundamentally natural-language artifacts that contains information that may not be captured by structured repository signals alone. Issue descriptions may implicitly reflect implementation complexity, ambiguity, urgency, dependency conflicts, or coordination overhead. Advances in natural language processing, specifically transformer-based language models such as DeBERTa (He et al., 2021), have substantially improved representation learning for unstructured text across a wide range of prediction tasks. Simultaneously, the "text-as-data" paradigm has increasingly encouraged language to be treated not merely as auxiliary context, but as a primary predictive signal in computational analysis tasks (Gentzkow et al., 2019).

Recent industrial and academic efforts have increasingly focused on repository-level software engineering agents capable of issue understanding, code modification, and multi-step reasoning over large repositories (Jimenez et al., 2024). This motivates hybrid modeling approaches that combine structured signals with semantic representations derived from issue text. In our case, the hybrid pipeline can be interpreted as a lightweight form of semantic augmentation, where a larger language model externalizes issue characteristics into structured summaries that is consumed by downstream transformer encoder. Such combinations may better

[1]New York University. Correspondence to: Eric Huang <zh2312@nyu.edu>, Bruce Zhang <yz8063@nyu.edu>.

Preprint. May 8, 2026.

capture both the organizational characteristics of repositories and the semantic complexity of issue discussions.

In this work, we investigate GitHub issue resolution time prediction using three progressively richer representation regimes: (1) shallow lexical and repository-level baselines using logistic regression, (2) transformer-based contextual text representations built on DeBERTa, and (3) a hybrid semantic-augmentation framework that prepends LLM-generated semantic summaries to the original issue text prior to transformer encoding. We evaluate these approaches under a temporal split setting designed to better reflect realistic deployment conditions. Our goal is not only to compare predictive performance, but also to examine how different forms of semantic representation affect issue resolution prediction under large-scale, heterogeneous GitHub environments.

2. Method

We investigate issue resolution prediction under three progressively richer representation regimes: structured repository signals, contextual language representations, and hybrid semantic augmentation. We instantiate these regimes as five models: M1 uses TF-IDF issue text only; M2 adds repository-level structured signals; M3 fine-tunes DeBERTa on issue text only; M4 adds structured signals to the DeBERTa setting; and M5 prepends LLM-generated semantic summaries to the M4 input. All models are trained as four-way classifiers over the resolution tiers defined in Section 3.

2.1. Structured Signal Baselines

Our first class of models uses only structured repository and issue metadata. These models represent conventional repository-mining approaches in prediction tasks.

Issue text fields are converted into sparse bag-of-words features using TF-IDF vectorization over concatenated `title` and `body` fields. Structured repository signals described in Section 3 are appended as dense numerical features after log scaling and standardization. Categorical variables such as `author_association` are one-hot encoded.

We train multinomial logistic regression classifiers using online `partial_fit` updates to support streaming over the full training partition. The resulting model serves as a lightweight baseline emphasizing explicit repository activity statistics and shallow lexical signals rather than deep semantic representations.

2.2. Transformer-based Text Modeling

Our second class of models replaces sparse lexical features with contextual language representations derived from DeBERTa-v3-base (He et al., 2021). Unlike TF-IDF features, transformer representations can encode semantic relationships, contextual phrasing, and implicit issue characteristics such as ambiguity, implementation complexity, and coordination difficulty.

Each issue is represented as the concatenation of its `title` and `body`. The transformer encoder is fine-tuned end-to-end using cross-entropy loss with class-balanced weights.

We treat this model family as a test of whether semantic information embedded in issue discussions provides predictive value beyond handcrafted repository signals alone.

2.3. Hybrid Semantic Augmentation

Our hybrid framework combines contextual issue text with structured semantic summaries generated by an external language model. Rather than directly predicting resolution labels, the summarization model extracts intermediate semantic attributes intended to expose latent issue characteristics in a more structured form.

For each issue, Qwen3.5-2B generates a five-field semantic summary containing: (1) issue type, (2) specification quality, (3) estimated implementation complexity, (4) urgency indicators, and (5) a short rationale regarding expected resolution speed. The summaries are prepended to the original issue text before DeBERTa tokenization.

This formulation differs from conventional ensembles. Instead of combining predictions from separate models, the hybrid pipeline injects language-model-derived semantic structure directly into the transformer input representation. The resulting model tests whether explicit semantic abstraction can improve downstream resolution prediction beyond raw issue text alone.

3. Data & Experiment Setup

3.1. Dataset and Label Construction

We construct our dataset from GH Archive,[1] a public log of all GitHub event streams queryable via Google BigQuery. We extract `IssuesEvent` records spanning a whole year (January 2025 to December 2025) across all public repositories.

Resolution time is defined as the number of elapsed calendar days between `created_at` and `closed_at`, and bucketed into four tiers: **Fast** ($\leq$7 days), **Medium** ($\leq$30 days), **Slow** ($\leq$90 days), and **Stale** ($>$90 days). Issues closed with `state_reason = not_planned` are unconditionally assigned to Stale regardless of elapsed time, as they represent maintainer rejection rather than resolution.

[1] https://www.gharchive.org/

This override prevents rejection events from appearing as spuriously fast resolutions.

We exclude bot-operated repositories receiving at least 100 issues per day on average, as these exhibit degenerate triage patterns inconsistent with human decision-making. After filtering, the dataset contains 8.69 million labeled issues. To mitigate label truncation bias, we discard issues filed after November 1, 2025, since Slow and Stale labels require up to 90 days to materialize after filing.

Table 1 reports the label distribution across splits. Contrary to the initial expectation that most open-source issues go unresolved, the training set is Fast-heavy. The test period shows a more pronounced shift toward Fast (69.7%), suggesting temporal drift in resolution behavior between the two periods. The minority classes Medium and Slow together account for only 21% of training examples, suggesting a class imbalance issue.

Table 1. Label distribution by split. Stale includes all issues closed as `not_planned` regardless of resolution time.

Class	Definition	Train	Test
Fast	$\leq$7 days	57.0%	69.7%
Medium	$\leq$30 days	13.0%	13.9%
Slow	$\leq$90 days	8.0%	7.5%
Stale	>90 days / rejected	22.0%	8.9%

3.2. Feature Set

We construct a two-level feature hierarchy using only information available at the moment an issue is filed.

Issue-level features: Each issue contributes three fields from the filing event: the `title` (free-form text summarizing the issue), the `body` (extended description), and `author_association`, a categorical variable encoding the filer's relationship to the repository (`OWNER`, `MEMBER`, `CONTRIBUTOR`, `NONE`, and others). Approximately 26% of test-set issues have a null `author_association`, treated as an implicit all-zeros category in one-hot encoding.

Repository-level signals: We augment each issue with five structured signals derived from repository activity events (`PullRequestEvent`, `ReleaseEvent`, `WatchEvent`, `PushEvent`). Each signal is computed using only events with `event_date` < `issue.created_at`, enforced via BigQuery window functions. The five signals are: PR merge rate (30-day rolling window prior to filing, as a proxy for maintainer responsiveness), median PR time-to-merge (30-day window, in hours), release cadence (release count over the preceding 90 days), star velocity (new stars in the preceding 30 days), and commit frequency (push events in the preceding 30 days). All five are log1p-scaled before use. Signal coverage is sparse in practice: PR signals are available for only 44% of issues, release signals for 24%, and star signals for 35%, with commit frequency the densest at 76%. This sparsity, reflecting the heterogeneity of the broader GitHub ecosystem, directly limits the marginal benefit of structured signals and is discussed further in Section 4.

LLM semantic summaries (Model 5 only). For our hybrid model, each issue additionally receives a structured semantic summary produced by Qwen3.5-2B (Qwen Team, 2026) in zero-shot inference. The prompt instructs the model to extract five structured fields from the issue text: issue type, specification quality, complexity, urgency signals, and a one-sentence resolution-speed rationale. The model receives only `title` and a truncated `body` (at most 1,000 characters), and the prompt explicitly prohibits predicting resolution-tier labels or referencing any post-submission information. The resulting summary is prepended to the raw issue text as `[SUMMARY: {summary}] [ISSUE: {title} {body}]` before tokenization, with the total input capped at 512 tokens by truncating the body first. Summaries are generated offline and cached prior to classifier training.

3.3. Evaluation Setup

Temporal split: We use a temporal rather than random split to simulate realistic deployment conditions. Models are trained on issues filed before August 1, 2025, and evaluated on issues filed between August 1 and October 31, 2025. Random splits would allow future repository patterns to contaminate training through issues from the same repository and period, inflating held-out performance. For the logistic regression models (Models 1 and 2), the full training partition is processed in a streaming fashion. For the DeBERTa models (Models 3 and 4), we draw a 500,000-example stratified sample from the training partition to fit within GPU memory constraints. For the hybrid model (Model 5), Qwen inference cost limits the labeled set to 40,000 training and 10,000 test examples, sampled proportionally across all source shards.

Fair comparison for Model 5: Because the Model 5 evaluation subset differs from the full test set used for Models 3 and 4, we re-evaluate Model 4 on the identical 10,401 Qwen-covered test rows. All comparisons between Models 4 and 5 use this matched subset.

Evaluation metric: We report macro-averaged F1 as the primary metric. Accuracy is misleading under the observed class imbalance, particularly given the 57% Fast majority in training. Macro-F1 weights all four classes equally, penalizing models that systematically ignore the minority Medium and Slow classes. Per-class F1 and confusion matrices are reported as secondary diagnostics in Section 5.

Class imbalance: All models apply class-weighted loss to counteract the Fast majority. For the logistic regression models, per-sample weights computed via `compute_class_weight("balanced")` are passed to each `partial_fit` call. For the DeBERTa models, the same weights are passed to `CrossEntropyLoss`. We do not apply oversampling or threshold calibration, as our goal is to assess the representations themselves rather than post-hoc decision boundary adjustment.

4. Results

4.1. Overall Performance

Table 2 reports macro-F1 and per-class F1 scores for all five models on the full temporal test set (August–October 2025). For Model 5, which is evaluated on the smaller Qwen-covered subset, we include Model 4 re-evaluated on the identical 10,401 test rows as a paired baseline in Table 3.

4.2. Key Findings

Fine-tuning is the dominant driver of improvement: Replacing TF-IDF with a fine-tuned DeBERTa encoder demonstrates the largest single gain in the ladder. This improvement is consistent across all four classes, with the most pronounced gains on Medium (+0.035) and Slow (+0.124), the two classes that TF-IDF features handle most poorly. **Structured repository signals provide consistent but marginal gains:** Adding the five repo-level signals improves macro-F1 by +0.002 over the LR text baseline (Models 1 to 2) and by +0.005 over the DeBERTa text baseline (Models 3 to 4). This modest benefit is consistent with the high signal sparsity reported in Section 3: PR and release signals are absent for the majority of issues

Qwen semantic summaries do not improve over the raw-text baseline: On the matched Qwen-covered test subset, Model 5 achieves a macro-F1 of 0.357 compared to 0.361 for Model 4†, a difference of -0.004. The hybrid model slightly improves Fast F1 (+0.003) but degrades Slow and Stale F1, suggesting that the structured summaries may shift the model's attention toward common, easier cases while losing signal for ambiguous long-tail classes. We discuss potential explanations, including the reduced body context available to DeBERTa under the hybrid input format, in Section 5.

Medium and Slow remain consistently difficult: Across all five models, the Medium and Slow classes exhibit the lowest per-class F1 scores. Even the best model achieves only 0.272 on Medium and 0.214 on Slow. These two classes occupy the interior of the resolution-time spectrum (8–90 days), where the boundary between them is defined by a hard cutoff rather than a natural discontinuity in maintainer behavior. This confusion is examined further in Section 5.

5. Analysis & Discussion

We conduct a detailed post-hoc error analysis on a 2,024-row diagnostic subset drawn from the Qwen-covered temporal test population, enabling paired qualitative comparisons across Models 4 and 5.

5.1. Class-wise Failure Modes

Table 3 shows row-normalized confusion matrices for both models. Fast and Stale achieve reasonable on-diagonal rates (58.4%/58.9% and 44.3%/44.3%, respectively), but Medium and Slow collapse almost uniformly across all four classes. In Model 4, true Slow predictions are distributed as 22.8% Fast, 22.0% Medium, 31.7% Slow, and 23.6% Stale, near uniform. Model 5 is equally diffuse.

This reflects the nature of the class boundaries: the 7- and 30-day thresholds separating Fast from Medium and Medium from Slow are hard calendar cutoffs, not natural discontinuities in maintainer behavior. Issues resolved on day 8 versus day 6 are fundamentally similar in content and repository context; correctly separating them requires information that is weak or unavailable at filing time.

5.2. Repository Activity as a Proxy for Fast Resolution

The model learns to use recent repository activity as a proxy for resolution speed. Correctly predicted Fast issues exhibit a median `push_count_30d` of 37 (Model 4) versus a median of 10 for true-Fast issues incorrectly predicted non-Fast (Model 5: 35 vs. 11). PR merge rate and release cadence show similar patterns.

The model learns a simple activity-based heuristic: repositories with higher recent activity are associated with faster issue resolution. This heuristic, however, breaks down near the Fast/non-Fast boundary. False Fast errors (non-Fast issues predicted Fast) are overrepresented among unaffiliated authors (`author_association=NONE`: $+18.5$ percentage points above the overall baseline in Model 4, $+14.4$ points in Model 5) and among issues with 0–30 words ($+5.9$ points). These patterns suggest the model conflates unaffiliated, short issues in active repositories with quick resolution, when many such issues actually belong to automated or educational workflows that close for administrative rather than triage reasons.

5.3. Text Complexity and Semantic Error Patterns

Text length functions as a proxy for implementation scope. True-Fast issues incorrectly predicted non-Fast have a median word count of 53, versus 22 for correctly predicted Fast issues, which is a pattern consistent across both models.

Table 2. Per-class and macro-averaged F1 scores. Models 1–4 are evaluated on the full test set (1,351,901 issues). Model 5 and its paired baseline (Model 4†) are evaluated on the matched 10,401 Qwen-covered test rows.

	Model	Fast	Medium	Slow	Stale	Macro-F1
M1	LR, text only	0.607	0.191	0.114	0.282	0.298
M2	LR, text + signals	0.650	0.237	0.066	0.250	0.301
M3	DeBERTa, text only	0.657	0.272	0.190	0.306	0.356
M4	DeBERTa, text + signals	0.679	0.255	0.204	0.306	0.361
M4†	DeBERTa, text + signals (subset)	0.672	0.262	0.214	0.294	0.361
M5	Qwen + DeBERTa, text + signals (subset)	0.675	0.268	0.195	0.287	0.357

Table 3. Row-normalized confusion matrices on the 2,024-row diagnostic subset. Columns are predicted class (F = Fast, Me = Medium, Sl = Slow, St = Stale); diagonal entries are **bold**.

True \ Pred	F	Me	Sl	St
Model 4 — DeBERTa, text + signals				
Fast	**.584**	.192	.093	.130
Medium	.227	**.339**	.268	.166
Slow	.228	.220	**.317**	.236
Stale	.282	.174	.101	**.443**
Model 5 — Qwen + DeBERTa, text + signals				
Fast	**.589**	.229	.058	.124
Medium	.261	**.390**	.193	.156
Slow	.211	.309	**.268**	.211
Stale	.268	.221	.067	**.443**

Longer issues are more likely to describe multi-step feature requests or open-ended investigations, both of which legitimately correlate with slower resolution. Fast precision is high (0.861 for Model 4, 0.857 for Model 5), but Fast recall is only 0.584 and 0.589, respectively. This analysis reveals our models are correctly conservative, avoiding the label for complex issues, but miss a substantial fraction of genuinely short-cycle tickets.

5.4. Qwen Summaries: Interpretability Without Predictive Gain

Model 5's Qwen-derived fields expose interpretable structure in the errors without improving predictive separation. True-Fast issues incorrectly predicted non-Fast are overrepresented in `qwen_urgency=moderate` (+8.3 percentage points above baseline) and `qwen_issue_type=feature` (+6.0 pp): Qwen characterizes these issues as moderate-priority feature requests, which the model pessimistically labels non-Fast. In practice, many resolve quickly because they are small-scope tasks in active teaching or sprint repositories.

Conversely, false Fast errors are overrepresented in `qwen_spec_quality=partial` (+7.1 pp). The model interprets partial specification as a signal of simple, implementable scope — a plausible heuristic that fails for delayed issues whose ambiguity reflects coordination difficulty rather than low implementation effort.

On the matched 10,401-row evaluation, Model 5 trails Model 4† by 0.004 macro-F1. The Qwen summaries provide structured semantic characterization that aids interpretability, but their structured fields do not reliably predict the specific 7/30/90-day tier boundaries. The hybrid input format also compresses the available body context within the 512-token DeBERTa budget, which may further limit recall on the Slow and Stale classes.

5.5. Limitations

Input truncation under the hybrid format: DeBERTa-v3-base accepts at most 512 subword tokens per input. For Models 3 and 4, the concatenated title and body are truncated at this limit, which silently discards the tail of long issue descriptions. In Model 5, the prepended Qwen summary consumes approximately 100 tokens of this budget, further compressing the body to roughly 400 tokens before truncation. Issues with detailed reproduction steps, extended stack traces, or long discussion threads are therefore more aggressively cut in the hybrid setting. We partially address this by warm-starting Model 5 from the saved Model 4 checkpoint rather than from the base DeBERTa weights: because the encoder has already learned attention patterns calibrated to this truncated issue distribution, the Model 5 fine-tuning begins from a more informed representation and does not need to relearn basic domain priors from a more compressed input signal. However, warm-starting does not recover the discarded tokens, and the net effect of the reduced body window likely explains part of the Slow and Stale recall degradation observed in Model 5.

Arbitrary resolution tier boundaries: The four-class label scheme partitions resolution time at 7, 30, and 90 days. These thresholds were set by judgment rather than derived empirically from the distribution of resolution times in the data. We did not conduct a survival analysis or cluster the observed resolution-day distribution to identify natural breakpoints, so it is possible that the boundaries do not correspond to behaviorally meaningful transitions in main-

tainer triage behavior. In particular, the Medium and Slow tiers — issues resolving in 8–30 days versus 31–90 days — show near-identical confusion patterns across all models, which is consistent with the hypothesis that no reliable signal distinguishes them at filing time. Alternative binning strategies, such as a three-class scheme merging Medium and Slow, or data-driven quantile thresholds, may produce a more learnable label space and are a natural direction for future work.

5.6. Summary of Interpretable Findings

Taken together, our error analysis surfaces four interpretable findings about what drives issue resolution speed in practice.

Repository maintenance tempo is the strongest filing-time predictor of fast resolution. Issues filed in repos with high recent commit activity (`push_count_30d` median of 37 for correctly predicted Fast issues versus 10 for missed Fast issues) and faster PR merge rates are far more likely to resolve quickly. This reflects a simple underlying dynamic: maintainers who are actively committing and merging pull requests are also more likely to triage new issues promptly. Repo activity signals are sparse across the broader GitHub population, which directly limits how much models can exploit this relationship.

Issue text length is a reliable proxy for implementation scope. Correctly predicted Fast issues have a median length of 22 words; true-Fast issues the model fails to catch have a median of 53 words. Short, focused issues tend to describe narrow, actionable problems that maintainers can resolve without extended deliberation. Longer issues more commonly describe multi-step investigations, architectural discussions, or feature requests requiring broader consensus, all of which naturally extend resolution time. This relationship holds consistently across both Models 4 and 5.

Contributor status interacts with issue complexity to predict delays. Unaffiliated authors (`author_association=NONE`) filing short issues are disproportionately responsible for false Fast predictions: these look like simple, quick-turnaround tickets but resolve slowly, likely because unaffiliated contributors lack the social standing to get prompt maintainer attention regardless of issue scope. Conversely, issues filed by owners and collaborators are easier to classify correctly, as established contributors tend to file tighter, more actionable reports and receive faster responses from the same maintainer community.

The Medium and Slow classes are not separable at filing time. This is the most consequential finding. Across both models, true Medium and true Slow predictions scatter near-uniformly across all four predicted classes, with no model achieving reliable recall on either tier. The 30-day boundary separating them is an arbitrary calendar threshold rather than a natural discontinuity in maintainer behavior. No combination of text, repository signals, or LLM-generated semantic summaries reliably places an issue on one side of this boundary at filing time, which explains why macro-F1 remains bounded despite progressively richer representations. Resolving this likely requires either post-submission signals or a coarser label scheme that merges the two interior classes.

6. Conclusion

We studied GitHub issue resolution-time prediction under a temporal split using structured repository signals, DeBERTa-based text models, and LLM-generated semantic augmentation. Our results show that fine-tuned contextual text representations substantially outperform shallow lexical baselines, while repository-level activity signals provide only marginal gains due to sparse coverage. Contrary to our initial expectation, LLM-generated semantic summaries improve interpretability but do not improve macro-F1 over the raw-text DeBERTa baseline. Error analysis suggests that repository activity, issue length, and contributor status are meaningful filing-time signals, but the Medium and Slow classes remain difficult to distinguish under the current 7/30/90-day label scheme. These findings suggest that future work should explore coarser or data-driven resolution bins, post-submission signals, and more targeted semantic augmentation strategies.

Code Access

Our implementation, preprocessing pipeline, and experiment scripts are available at: https://github.com/cocoa-huang/gh-issue-resolution.

References

Gentzkow, M., Kelly, B., and Taddy, M. Text as data. *Journal of Economic Literature*, 57 (3):535–74, September 2019. doi: 10.1257/jel.20181020. URL https://www.aeaweb.org/articles?id=10.1257/jel.20181020.

Gousios, G., Pinzger, M., and Deursen, A. v. An exploratory study of the pull-based software development model. In *Proceedings of the 36th International Conference on Software Engineering*, ICSE 2014, pp. 345–355, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450327565. doi: 10.1145/2568225.2568260. URL https://doi.org/10.1145/2568225.2568260.

He, P., Liu, X., Gao, J., and Chen, W. Deberta: Decoding-enhanced bert with disentangled attention, 2021. URL https://arxiv.org/abs/2006.03654.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. The promises and perils of mining github. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, MSR 2014, pp. 92–101, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450328630. doi: 10.1145/2597073.2597074. URL https://doi.org/10.1145/2597073.2597074.

Qwen Team. Qwen3.5-omni technical report. *arXiv preprint arXiv:2604.15804*, 2026.

A. Additional LLM Baseline Results

We additionally explored prompting-based large language model baselines to evaluate whether modern LLMs can directly infer issue resolution tiers from raw issue text without supervised fine-tuning.

Specifically, we evaluated DeepSeek-v4-Flash and GPT-5.4-mini under both zero-shot and few-shot prompting settings. In the zero-shot setting, the model receives only the issue title and body together with the resolution-tier definitions. In the few-shot setting, we prepend several labeled demonstration examples sampled from the training split before inference.

Table 4 summarizes the results under the same temporal evaluation setting used throughout the paper.

Table 4. Prompting-based LLM baselines under the temporal split setting.

Model	Prompting	Fast	Medium	Slow	Stale	Macro-F1
DeepSeek-v4-Flash	Zero-shot	0.652	0.241	0.127	0.138	0.290
GPT-5.4-mini	Zero-shot	0.596	0.257	0.136	0.182	0.293
GPT-5.4-mini	Few-shot	0.550	0.305	0.149	0.121	0.281
DeepSeek-v4-Flash	Few-shot	0.528	0.293	0.195	0.073	0.273

Overall, prompting-based approaches underperform the fine-tuned DeBERTa models reported in the main text. Zero-shot prompting achieves macro-F1 scores around 0.29, while few-shot prompting does not consistently improve performance and in some cases slightly degrades it. A possible explanation is that GitHub issue resolution behavior depends heavily on repository-specific maintenance dynamics and temporal activity patterns that are difficult to infer purely from a small number of in-context demonstrations. These results suggest that while modern LLMs can capture coarse semantic characteristics of issue discussions, supervised adaptation remains more effective for large-scale resolution-time prediction under heterogeneous repository environments.